

UNIVERSIDAD DE CHILE
FACULTAD DE CIENCIAS FÍSICAS Y MATEMÁTICAS
DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN

EXPLORANDO LA OPTIMIZACIÓN DEL ENTRENAMIENTO DE DISPOSITIVOS BCI, EN BASE A TÉCNICAS DE CLUSTERING Y MACHINE LEARNING

MEMORIA PARA OPTAR AL TÍTULO DE
INGENIERO CIVIL EN COMPUTACIÓN

MÁXIMO CLEMENTE DESIDERIO RETAMAL ROJAS

PROFESOR GUÍA:
NELSON BALOIAN T

MIEMBROS DE LA COMISIÓN:
IVANA BACHMANN
PATRICIO INOSTROZA F.

SANTIAGO DE CHILE
2026

Resumen

La interfaz cerebro-computador (*BCI*) ha emergido como una tecnología prometedora que establece una conexión directa entre el cerebro humano y un computador, permitiendo la comunicación y control de dispositivos a través de la actividad cerebral. Este campo de investigación ha generado un gran interés debido a su potencial para mejorar la calidad de vida de personas con discapacidades motoras, así como para ampliar las capacidades de interacción hombre-máquina en diversos campos.

El presente trabajo aborda el desarrollo y evaluación de una aplicación de *BCI* diseñada para entrenar comandos mentales utilizando el dispositivo *Epoch +*. El objetivo de este trabajo es explorar métodos que permitan agrupar personas de forma que el proceso de entrenamiento de una persona permita acelerar el de otra.

En este informe, se describirán en detalle los aspectos técnicos de la aplicación, incluyendo el proceso de entrenamiento de los comandos mentales, la normalización de datos y la metodología utilizada para la evaluación de la precisión del reconocimiento de comandos. Además, se presentarán los resultados obtenidos durante las pruebas experimentales, seguidos de un análisis y discusión detallados.

Agradecimientos

Quisiera expresar mi sincero agradecimiento, en primer lugar, a mi familia, por su apoyo incondicional durante todos estos años de estudio. Su aliento y comprensión fueron fundamentales para mantenerme enfocado y superar los desafíos que se presentaron en el camino.

Agradezco también a mis amigos y compañeros, quienes compartieron conmigo esta etapa de aprendizaje y crecimiento. Sus palabras de aliento y compañía hicieron que este viaje fuera más llevadero y memorable.

Por último, pero no menos importante, quiero agradecer a los profesores que me guiaron y enseñaron a lo largo de estos años. Su dedicación y conocimiento fueron una inspiración constante, y sin su orientación, seguramente la historia habría sido diferente.

A todos ustedes, mi más profundo agradecimiento por haber sido parte de este camino y haberme ayudado a alcanzar este logro.

Tabla de Contenido

1. Introducción **1**

2. Estado del arte **3**

2.1. Interfaz Cerebro-Computador 3

2.2. Sobre el entrenamiento 4

2.3. Dispositivo de adquisición de señal 4

2.3.1. El Sistema 10-20 5

2.3.2. Flujo de uso del dispositivo 7

2.4. Ondas cerebrales 7

2.5. Redes Neuronales 8

2.6. Clustering. 9

2.6.1. Maldición de la dimensionalidad 10

2.6.2. Feature selection vs Feature transformation 11

2.6.3. Número de clusters 11

2.6.4. Evaluación de Clusters 11

2.6.5. Deep Clustering 13

2.6.6. Autoencoder 14

2.6.7. Enfoques de deep-Clustering 15

3. Enfoque **17**

4. Adquisición de datos **20**

4.1. Aplicación 20
4.1.1. Parte visual de la aplicación 21
4.1.2. Parte Encargada de entrenar comandos 21
4.1.3. Recolección de los datos 22
4.1.4. Resultados de la prueba 22
4.1.5. Diferencia entre aplicación y software de *Emotiv* 23

5. Análisis **25**
5.1. Búsqueda clusters 25
5.1.1. Reducción de dimensiones con Autoencoder 25

6. Conclusión **28**

Bibliografía **31**

Apéndice A. Anexo **32**
A.1. Imágenes aplicación entrenamiento 32

Índice de Tablas

2.1. diferentes bandas de ondas cerebrales y sus rangos de frecuencia asociados. . 8

3.1. Distintas formas de agrupar datos 18

Índice de Ilustraciones

2.1. Dispositivo *EPOC +*. 5

2.2. Posición de electrodos en el sistema 10-20 6

2.3. Posición de electrodos de *EPOC +* según el sistema 10-20 6

2.4. Diagrama sistema 10-20. [29] 6

2.5. Sistema 10-10. 7

2.6. Encefalograma humano. 8

3.1. Representación de entrenamientos 18

4.1. Promedio de proporción de comandos correctos por ronda 23

4.2. Entrenamiento de comando mental a través de *EnotivBCI*. 24

4.3. Entrenamiento de comando mental a través de aplicación desarrollada. . . . 24

4.4. Comparación visual de aplicaciones de entrenamiento. 24

5.1. Resultado reducción dimensionalidad con AE y PCA sobre datos *pow* utilizando datos de la forma humano-comando 26

5.2. Resultado reducción dimensionalidad con AE y PCA sobre datos *pow* utilizando datos de la forma humano-ronda 26

5.3. Resultado reducción dimensionalidad con AE y PCA sobre datos pow utilizando datos de la forma humano *pow* 27

A.1. Mensaje al iniciar la aplicacion 33

A.2. Mensaje presente al entrenar comando neutral 33

A.3. inicio y fin de animación presente al entrenar comando *"empujar"*. 34

A.4. inicio y fin de animación presente al entrenar comando *"tirar"*. 34
A.5. inicio y fin de animación presente al entrenar comando *"caer"*. 35
A.6. inicio y fin de animación presente al entrenar comando *"levantar"*. 35
A.7. inicio y fin de animación presente al entrenar comando *"izquierda"*. 36
A.8. inicio y fin de animación presente al entrenar comando *"derecha"*. 36
A.9. Mensaje presente al terminar el proceso 37

Capítulo 1

Introducción

El mundo está cada vez más conectado a través de computadores, los cuales manejan una cantidad creciente de información. Sin embargo, la interacción entre humanos y computadoras se ve limitada por la velocidad de movimiento de los dedos, que está determinada biológicamente y puede ser especialmente baja para personas con movilidad reducida. Estas personas se beneficiarían de una forma de interactuar con las computadoras que no requiera movimiento.

Una forma de superar esta barrera es establecer una conexión directa entre el cerebro humano y un computador. Esto se logra mediante una interfaz cerebro-computador (*BCI*, por sus siglas en inglés).

La tecnología *BCI* permite una comunicación directa entre el cerebro y un computador. Por lo general, esta comunicación se realiza midiendo y analizando la actividad cerebral, que luego se traduce en comandos comprensibles por el computador.

En su nivel más básico, los sensores utilizados realizan mediciones de la actividad eléctrica del cerebro a través de una electroencefalografía (*EEG* por sus siglas en inglés), la cual registra los electrogramas de las actividades eléctricas espontáneas del cerebro. Estos datos se convierten en información numérica, que debe ser "traducida" a instrucciones comprensibles para el computador. Cada persona medida requiere un proceso de entrenamiento individualizado para lograr esta traducción, este proceso es lo que se llamará más adelante "entrenamiento", el cual demora un tiempo considerable, tiempo que se buscará reducir.

Históricamente, la tecnología *BCI* se ha utilizado principalmente en el ámbito médico, especialmente en pacientes con problemas de movilidad. En este entorno, el tiempo de entrenamiento no es un factor crítico. Sin embargo, en los últimos años, con la reducción de costos y el tamaño de los sensores, estos dispositivos han ingresado en el mercado en mayor cantidad, lo que ha creado nuevas posibilidades de uso, donde el tiempo de entrenamiento ha pasado a ser un obstáculo para su adopción.

En los últimos años, han surgido dispositivos *BCI* con menor resolución en comparación con los dispositivos utilizados anteriormente. Esta reducción de resolución también ha implicado una disminución en el costo, lo que ha permitido explorar problemas que no requieren

mediciones de alta precisión, como la escritura sin teclado [24, 32] o su uso en videojuegos [31]..

A pesar de la reducción de costos y el aumento en la cantidad de publicaciones [26], aún es común encontrar problemas sin resolver, por ejemplo, está el problema de entrenamiento, creado por las diferencias biológicas existentes entre los cerebros humanos, las cuales provocan la necesidad de comenzar el uso del *BCI* repitiendo comandos hasta lograr reconocer el cómo se registra la acción en el cerebro específico. Este entrenamiento suele durar 20-30 minutos y solo es necesario la primera vez que se utiliza el *BCI* específico.

Aunque el tiempo requerido para la realización del entrenamiento ha disminuido a lo largo del tiempo, sigue siendo un obstáculo para una adopción masiva de esta tecnología. Por lo tanto, se plantea la búsqueda de patrones repetitivos entre individuos que permitan agruparlos, de modo que el entrenamiento de un miembro de un grupo ayude a reducir el tiempo requerido para entrenar el software cuando se utiliza por primera vez con un usuario nuevo. Se desea investigar la existencia de estos grupos y para averiguar si es posible clasificar a nuevos usuarios y completar el entrenamiento en menos tiempo que el actual, manteniendo una precisión comparable.

La reducción en el tiempo de entrenamiento podría así eliminar una de las barreras para una mayor adopción de esta tecnología, lo que aumentará las posibilidades de alcanzar un futuro en el que sea posible comunicar grandes cantidades de información directamente desde el cerebro.

Dada la combinación del desarrollo de nuevas herramientas de clasificación y la existencia del problema de entrenamiento en *BCI*, se encuentra que hoy en día se puede abordar este problema con el enfoque planteado, aunque no se garantiza que los resultados vayan a ser positivos. Es por esto que el objetivo de este trabajo de título es averiguar la factibilidad o no de este enfoque, pudiendo obtener una respuesta tanto positiva como negativa.

Como ya se describió anteriormente, el fin de este trabajo es reducir este tiempo, para lo cual se hipotetiza que es posible encontrar personas tales que el resultado del entrenamiento de una de ellas pueda ser utilizada por por otras, acortando así el tiempo requerido por nuevos usuarios en el entrenamiento de comandos utilizables por dispositivos *BCI*.

El trabajo realizado, es un trabajo exploratorio, el cual busca definir el problema de manera que futuros trabajos puedan continuar la investigación y realizar una búsqueda exploratoria de soluciones al problema expuesto. Se definirá una forma de manejar los datos, crear una base de datos y se mostrarán resultados de la búsqueda de grupos.

Capítulo 2

Estado del arte

En esta sección se describe tanto el estado del arte en el ámbito de la tecnología *BCI*, incluyendo además el dispositivo utilizado en el estudio, ya que de ello depende la extracción de datos utilizados en la investigación. Además, se aborda la investigación sobre el estado del arte de *deep-clustering* y otras tecnologías relacionadas con la clusterización y redes neuronales.

2.1. Interfaz Cerebro-Computador

Las interfaces cerebro-computador, también conocidas como *BCI*, establecen una conexión entre el cerebro de una persona y un computador. Para lograrlo, en primer lugar, la interfaz utiliza sensores para registrar la actividad cerebral. Posteriormente, las señales registradas son sometidas a un preprocesamiento con el fin de eliminar ruido y artefactos. Luego, se lleva a cabo la extracción de características (*feature extraction*), donde se analiza la señal y se extrae información relevante para el proceso de clasificación, que determina el tipo de tarea que se debe realizar. Finalmente, se envía la instrucción asociada a la tarea a la aplicación o dispositivo correspondiente [21].

Existen a grandes rasgos 3 categorías de *BCI* en la manera en que se relacionan al cerebro. En primer lugar se encuentran los *BCI* pasivos, los cuales decodifican los estados afectivos y/o cognitivos no intencionales del cerebro, en contraste existen los *BCI* activos, los cuales trabajan con la actividad provocada por el accionar voluntario del usuario y se tienen además los *BCI* reactivos, los cuales trabajan con las respuestas generadas por estímulos externos al usuario. [26]

En términos del método de adquisición de señales, existen dos categorías principales: invasivo y no invasivo. Los *BCI* invasivos presentan una mejor relación señal/ruido, pero requieren ser implementados mediante una intervención quirúrgica. Debido a esta naturaleza invasiva, solo suelen ser utilizados en entornos médicos. Por otro lado, los BCI no invasivos, aunque son más susceptibles al ruido, no requieren una intervención quirúrgica y, por lo tanto, permiten una mayor variedad de aplicaciones en diversos contextos [26].

2.2. Sobre el entrenamiento

A continuación se detalla el objetivo principal de este trabajo: la reducción del tiempo de entrenamiento en el contexto de *BCI*. A lo largo de la investigación, se han abordado diversas estrategias para abordar esta problemática, logrando avances significativos en la reducción de la duración del proceso. Sin embargo, a pesar de estos logros, el problema aún no ha sido completamente resuelto, lo cual destaca su gran relevancia en el campo de estudio. A continuación, se presentan diferentes enfoques que se han explorado en la investigación de este problema.

Se han desarrollado métodos que emplean técnicas de clusterización en la optimización de los tiempos de reajuste en usuarios experimentados [17]. Este estudio reveló que los usuarios experimentados no necesitan volver a entrenar el dispositivo cada vez que lo utilizan. Aunque se concluyó que este enfoque no sería adecuado para usuarios novatos sin experiencia previa, es posible que los avances en clasificación y *machine learning* posteriores al momento en que se publicó el artículo permitan aplicar esta aproximación incluso en novatos.

Existen intentos modernos en la búsqueda de acelerar el entrenamiento de *BCI*, por lo general, los intentos utilizan deep learning [12, 9], utilizando en específico la utilización de *transfer learning* (parte de ML), con el propósito de transferir conocimiento entre sesiones/-sujetos, en estos estudios se trabajó directamente con mediciones *EEG* y utilizando diferentes variaciones de redes neuronales consiguen resultados favorables.

Existe la idea de incluir mediciones *EEG* del cerebro mientras descansa a un modelo de clasificación utilizando *transfer learning* podría mejor reducir el tiempo de entrenamiento [25], teniendo el inconveniente de requerir gran cantidad de datos.

Otro enfoque posible es adaptar las nuevas mediciones a información conocida, asociada a mediciones anteriores, para luego elegir alguna sesión de entrenamiento completa y utilizar los datos adaptados sobre ella [5], este método podría ayudar al pre-entrenamiento, apoyando la toma de decisión a la hora de elegir el grupo a asignar.

2.3. Dispositivo de adquisición de señal

El trabajo se realizo con un *BCI* no invasivo de la marca *EMOTIV*, modelo *EPOC +*, [10] este dispositivo cuenta con 14 sensores de Electroencefalograma (*EEG*), cuyos sensores son del tipo salino, esto significa que el contacto entre el sensor y la piel se realiza con una esponja, la cual requiere estar humectada con una solución salina para su uso efectivo, la ventaja que entrega este tipo de sensor es reducción de impedancia asociada a la piel, comparado con sensores secos, mejorando así la relación de señal/ruido.

El dispositivo cuenta con aplicaciones gráficas [10], las cuales entregan funcionalidades básicas en el uso del *BCI*, teniendo principalmente la habilidad de realizar entrenamiento de comandos y comprobar la calidad de señal siendo recibida por los sensores, asegurando así un correcto uso del dispositivo. Además se cuenta con una api oficial [11], la cual permite crear aplicaciones con funcionalidades más variadas, principalmente se usará en la captura

del proceso de entrenamiento.



Figura 2.1: Dispositivo *EPOC +*.

Los 14 sensores del dispositivo representan los canales AF3, F7, F3, FC5, T7, P7, O1, O2, P8, T8, FC6, F4, F8 y AF4 del sistema 10-20. Este es un sistema internacional que indica las posiciones en la que colocan los electrodos de forma no invasiva.

2.3.1. El Sistema 10-20

El sistema 10-20 es un estándar de posicionamiento de electrodos, sobre el cráneo humano al medir señales *eeg*. Su nombre proviene de las distancias entre nodos adyacentes los cuales están a 10 % o 20 % de la distancia total del cráneo, esto es la distancia entre la parte frontal y la trasera del cráneo y la distancia entre el extremo izquierdo y el derecho. El estándar busca cubrir distintas áreas del cerebro y poder comparar resultados provenientes de distintos dispositivos y de cráneos con diferentes formas y tamaños.

Cada posición de nodo está identificada por al menos una letra y en la mayoría de los casos un número. Las letras están dadas por los lóbulos correspondientes a áreas en que se localiza el sensor. Siendo estos, el lóbulo Pre-Frontal (Fp o Af), frontal (F), parietal (P), occipital (O), temporal (T) y también existe un electrodo en la apófisis mastoides (A) . Se utiliza la letra "z" para indicar que el electrodo se ubica sobre el plano sagital (plano perpendicular al suelo que pasa por la mitad del cuerpo, dividiendo el cuerpo en lado derecho e izquierdo). Finalmente se utilizan números pares para identificar nodos en el lado derecho y número impares para el lado izquierdo. Con sus valores incrementando al alejarse del centro.

como se puede ver comparando las imágenes 2.2 y 2.3 *emotiv +* tiene electrodos no presentes en el sistema 10-20 (Fc5, Fc6, T7, T8, P7 y P8) pero estos si están presentes en el sistema 10-10, el cual es una extensión del sistema 10-20 en el cual todos lo electrodos están a 10 % de la distancia total del cráneo.

Esta extensión del sistema introduce las zonas temporal poles (TP), central poles (CP),

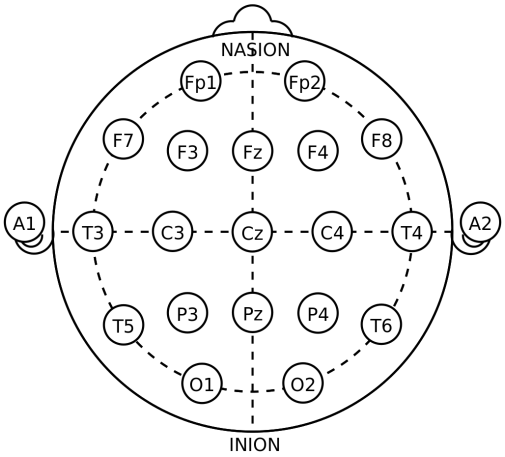


Figura 2.2: Posición de electrodos en el sistema 10-20

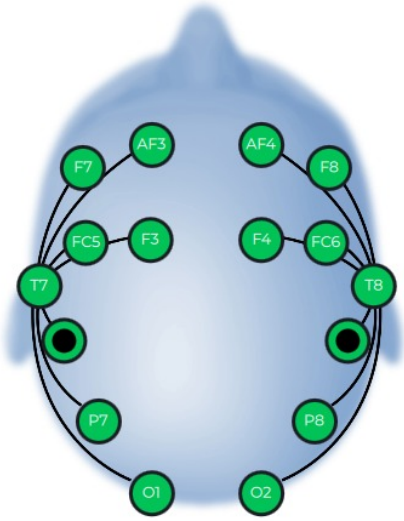


Figura 2.3: Posición de electrodos de *EPOC +* según el sistema 10-20

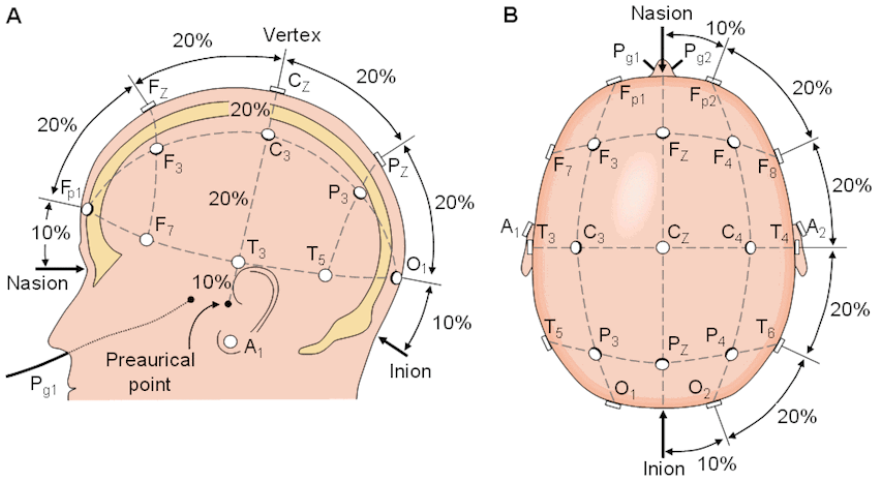


Figura 2.4: Diagrama sistema 10-20. [29]

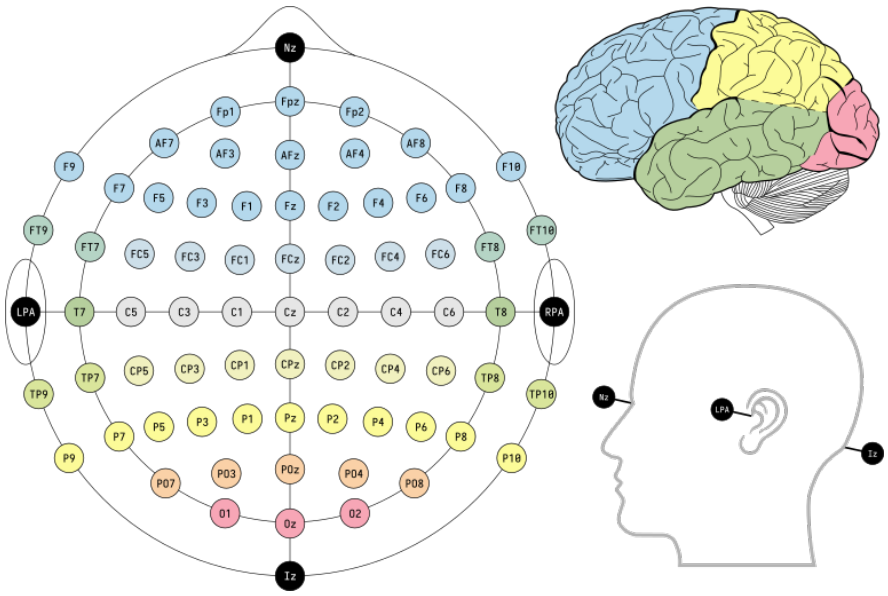


Figura 2.5: Sistema 10-10.

frontotemporal (FT) y parietooccipital (PO).

2.3.2. Flujo de uso del dispositivo

El uso normal del dispositivo, inicia con la hidratación de los electrodos, utilizando liquido salino. Además se conecta el dispositivo a un computador con el software *Emotiv Launcher*, software requerido para la conexión del dispositivo *BCI* con los servidores de *Emotiv* y aplicaciones hechas con la api de la misma empresa. Esta aplicación también indica la calidad de contacto de los electrodos contra la cabeza del sujeto que utilizara el dispositivo, esto es necesario para maximizar la calidad de las mediciones que se realizaran a continuación.

Teniendo ya la conexión establecida y el dispositivo en contacto con la piel de la cabeza empieza el flujo de las aplicaciones desarrolladas con la api *Cortex*, el cual empieza comprobando el id de la aplicación, del cliente

En el uso normal del dispositivo, las señales detectadas por los sensores del dispotivo *BCI* son enviadas directamente a servidores del fabricante, posteriormente se reciben *streams* de datos, se reciben los *streams* en que se haya realizados suscripción, que además estén permitidos por la licencia contratada.

2.4. Ondas cerebrales

En el funcionamiento normal del cerebro humano, existen corrientes eléctricas recorriendo las neuronas, estas corrientes producen campos eléctricos, los cuales son medidos por electrodos que pueden estar situados al interior del cráneo (eeg invasivo) o presionados contra la piel del cráneo (eeg no invasivo). Las mediciones realizadas por estos electrodos producen un

Nombre banda	Rango frecuencia [Hz]
Beta Low	12 - 16
Beta High	16 - 25
Alfa	8 - 12
Theta	4 - 8
Gamma	25 - 45

Tabla 2.1: diferentes bandas de ondas cerebrales y sus rangos de frecuencia asociados.

encefalograma.

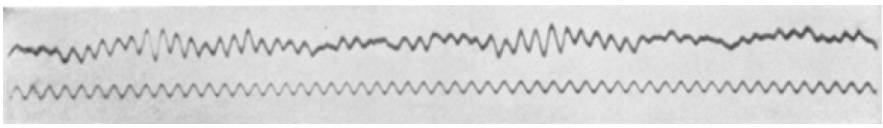


Figura 2.6: Encefalograma humano.

Cada electrodo define una curva del encefalograma, pero esta, por si sola no entrega suficiente información, es por esto que las ondas *EEG* se descomponen en distintas bandas de distintos rangos de frecuencia. en la tabla 2.1 se puede ver la descomposición de la onda utilizada por el dispositivo *EPOC +*, también existe la banda delta para frecuencias menores a 4 Hz y es común juntar ambas bandas Beta en una que abarque desde los 12,5 Hz a los 30 Hz pero las bandas descritas por la tabla 2.1 son las entregadas por la api de *Emotiv*.

El calculo de estas bandas es hecho en los servidores de emotiv, esto lo hacen utilizando transformadas rápidas de fourier.

La transformación realizada divide los 14 sensores en 5 ondas cada uno, por lo que se tienen 70 ondas por persona.

2.5. Redes Neuronales

Un enfoque de trabajo es aprovechar la secuencialidad en la que se originan los datos. Esto permite utilizar *Recurrent Neural Network* (RNN) y en particular *Long Short-Term Memory network* (LSTM) [27], las cuales tienen un mejor manejo de los gradientes, en comparación con las *RNN*. Estas redes ya han sido utilizadas sobre datos provenientes de *BCI*, donde por ejemplo ha sido utilizado en el reconocimiento de emociones [4].

Un enfoque popular en el mundo del lenguaje natural es el uso de *Transformers*, Este enfoque tiene algunas ventajas por sobre las redes recurrentes, tales como, la habilidad de ser entrenadas de forma paralela y una menor necesidad de datos. Centrándose en datos provenientes de *BCI*, se ha empleado *Transformers* para correlacionar información espacial con datos *EEG* [30], e incluso para decodificar tareas *BCI* sin realizar entrenamiento [18].

Algunas estructuras de redes neuronales utilizadas en Deep Learning

- **Deep Belief Networks** (DBN): son redes neuronales de aprendizaje no supervisado, en que algunas de las capas son "Restricted Boltzmann machine" lo cual es un tipo de red neuronal en la cual las neuronas forman un grafo bi-partito, esto significa que hay 2 grupos de nodos y solo existen conexiones intergrupales, la DBN junta varias de estas máquinas utilizando la salida de una como la entrada de la siguiente.
- **Convolutional neural network** (CNN): Es un tipo de red neuronal prealimentada, en la cual se tienen capas convolucionales, las cuales trabajan sobre tensores o matrices de datos, utilizan filtros (kernel) para reducir la dimensionalidad de los datos, estos filtros son matrices de menor dimensión pero misma "profundidad" al ser comparados con los datos originales, que se "deslizan" a lo largo de estos realizando una operación lineal sobre ambas matrices, consiguiendo así una matriz de menor dimensionalidad, llamada mapa de activación. Cada capa convolucional puede tener más de un kernel, por lo que el mapa de activación resultante tendrá una profundidad dependiente de la cantidad de filtros.

  Además de las capas convolucionales, existen capas de agrupamiento, las cuales reducen reducen dimensionalidad dividiendo la matriz de datos en submatrices de tamaños predefinidos, guardándose el promedio de los valores entrantes o el máximo entre ellos, siendo preferido el máximo, ya que reduce el ruido de los datos.
- **Long short-term memory** (LSTM): es un tipo de red neuronal recurrente que busca solucionar el problema de los gradientes desvanecientes, es además enfocado en el trabajo sobre datos con forma de series de tiempo, su nombre se debe a la incorporación de módulos de memoria los cuales permiten retener información pasada

2.6. Clustering.

Clustering es una forma de agrupar datos utilizando machine learning de forma no supervisada en grupos, esto en grupos no definidos previamente, es por esto que este es el principal enfoque del trabajo realizado.

El enfoque principal del trabajo es el agrupar personas de forma tal que personas pertenecientes a los mismos grupos puedan utilizar el entrenamiento de otros miembros del mismo grupo. Es por esto que el clustering tiene protagonismo en el trabajo, ya que el resultado de este coincide con lo buscado.

Algunos enfoques utilizados por algoritmos de clustering son:

- Enfocado en densidad: Trabaja detectando áreas con alta densidad de datos agrupándolos, separando los grupos por áreas de menor densidad.
- Enfocado en centroide: se define un cierto número de "centroides", los cuales actúan de centro de los clusters y el algoritmo los va desplazando en minimizando la distancia de los distintos puntos al centroide.
- Enfocado en distribución: Parecido al enfoque de centroide, pero la en este caso la distancia define la probabilidad que el punto pertenezca a el cluster, de acuerdo a alguna distribución probabilística.

- Enfocado en jerarquía: Típicamente utilizado en datos jerárquicos, se crea un árbol de clusters organizado desde la cima.
- Enfocado en grilla: Separa el espacio en el que se encuentran los datos en grillas y luego se realiza clustering sobre las grillas, esto permite trabajar sobre una cantidad mayor de datos de manera rápida.

El clustering de datos provenientes de dispositivos *BCI* no es algo nuevo, esto ya se ha hecho por ejemplo en el diagnóstico de tratamientos para la epilepsia [22]. También ha sido utilizado en la detección de conducción somnolienta [13]. Es por esto que se considera razonable el uso de este enfoque en el trabajo realizado.

La mayoría de los algoritmos de clustering trabajan sobre 2 dimensiones. Siendo *k-means* y *DBSCAN* ejemplos de algoritmos comúnmente utilizados. El problema de estos algoritmos, es la dimensionalidad con la que trabajan, siendo ésta muy inferior a la requerida para el trabajo. Es por esto que existe la opción de reducir la dimensionalidad utilizando algoritmos como *TSNE* o *PCA*, los cuales reducen la dimensionalidad a las 2 dimensiones requeridas por los algoritmos antes mencionados. O también se pueden utilizar algoritmos que trabajen sobre una mayor dimensionalidad de datos. El trabajar con datos de alta dimensionalidad introduce el problema de la maldición de la dimensionalidad, el cual se produce ya que las distancias en estos datos crea "confusión" en los algoritmos que dependen de esta.

Clusterizar en 2 dimensiones tiene sus desventajas al trabajar con datos de mayor dimensionalidad, esto debido a la pérdida de información producida al reducir la dimensionalidad de los datos, esto en datos eeg, elimina la efectividad de los algoritmos de clustering. Es por esto que se trabaja con algoritmos de clustering de alta dimensionalidad.

Un algoritmo de clustering de alta dimensionalidad es *CLIQUE*, este algoritmo tiene enfoque tanto de grilla como de densidad. Este funciona identificando cliques, los cuales son grupos de nodos en un grafo densamente conectados, luego define clusters como distintos grupos de cliques. Este método de clustering ya ha mostrado resultados en el clustering de datos *EEG* sin etiquetar [8].

2.6.1. Maldición de la dimensionalidad

La maldición de la dimensionalidad es un conjunto de fenómenos detectables al clusterizar (u otras actividades similares) datos de alta dimensionalidad, estos problemas dificultan el trabajo de algoritmos de clusterización; El mayor problema enfrentado es la pérdida de utilidad de la distancia entre puntos, esto debido a que en espacios de alta dimensionalidad los puntos tienden a estar equidistantes, esto es que para un número suficientemente alto de dimensiones, al elegir un punto cualquiera de los datos, su vecino mas cercano y su vecino más lejano estarán a distancias indistinguibles.[6], por lo que algoritmos dependientes de la distancia entre puntos pierde su utilidad.

Otros problemas comúnmente incluidos incluyen el aumento de la complejidad computacional, la tendencia a hacer overfitting ante la presencia de ruido y por último también es un problema la falta de intuición en humanos ante espacios de alta dimensión.

Al enfrentar la maldición de la dimensionalidad mientras se intenta clusterizar se tienen 2 formas de reducir el efecto de esta; se puede utilizar algoritmos hechos para trabajar con alta dimensionalidad o se puede reducir la dimensionalidad.

2.6.2. Feature selection vs Feature transformation

feature selection y feature transformation son 2 vías que se pueden tomar en la reducción de dimensionalidad la diferencia que existe entre ambas, es la conservación de atributos de los datos originales. Mientras feature transformation (Principal component analysis esta incluido aquí) conserva todos los atributos y generalmente conserva la distancia relativa entre objetos, esta área tiene la desventaja de mantener el efecto producido por ruido en los datos, esto es un problema en en datos ruidosos como los *eeg*.

Por otro lado, busca seleccionar los atributos de los datos que más aporten al trabajo realizado, para esto se requiere probar diferentes subconjuntos de los datos originales. Esto comúnmente se hace con modelos de "filtro", los cuales examinan la forma intrínseca de los datos, para luego evaluar subconjuntos de los features. También están los modelos de "wrapping", los cuales utilizan el modelo que requiere de la reducción de dimensionalidad para evaluar los distintos subconjuntos de los datos /citeparsons$_s ubspace_2$004.

2.6.3. Número de clusters

Existen maneras empíricas para decidir el número óptimo de clusters, el **método del codo**, el cual grafica una curva alguna métrica, comúnmente textbfSum of Squared Errors, al realizar algún algoritmo de clustering (comúnmente k-means) sobre los datos, buscando un punto donde la pendiente de la curva se vea reducida, este punto es el denominado "codo" y representa la cantidad de clusters, tal que el costo de aumentar el número de cluster por sobre este, tiene un mayor costo que el beneficio obtenido.

2.6.4. Evaluación de Clusters

Dada la alta dimensionalidad de los datos, el cuantificar la calidad de los clusters visualmente no es suficiente, además no se conocen las categorías de los datos por lo que métodos extrínsecos de calificación de clusters no son útiles para el problema, Debido a esto se utilizan métodos intrínsecos para calificar los clusters identificados.

Por lo general los métodos intrínsecos realizan a grandes rasgos el mismo cálculo, en la búsqueda de dar un valor numérico al grupo de clusters encontrados se calcula algún tipo de distancia que indique la densidad de cada cluster y se divide por algún valor referenciante a la distancia entre clusters. Algunos ejemplos que siguen este pharmaton incluyen:

- **Silhouette**: Una métrica intrínseca usualmente utilizada para identificar si los clusters son "válidos". Es el método de Silhouette el cual mide la similitud de los datos existentes

en clusters y la diferencia existente entre clusters, esto se hace calculando:

  - a_i: las distancias promedio de puntos con el resto de los puntos del mismo cluster.
  - b_i: la distancia promedio mínima del punto en referencia al punto (Se busca el cluster que en promedio está más cerca) distancia promedio al cluster mas cercano
  - se calcula:
$$\frac{b(i) - a(i)}{\max\{a(i), b(i)\}}$$
  - el resultado es el promedio habiendo calculado todos los puntos.

y la distancia promedio mínima del punto en referencia al punto (Se busca el cluster que en promedio está más cerca), teniendo ambos valores se asignando finalmente un valor entre -1 y 1 siendo un valor cercano a 1, una indicación de que los valores están bien asignados. Este método se calcula utilizando distancia entre datos, por lo que es susceptible a la maldición de la dimensionalidad.

- **Davies–Bouldin**: Un método utilizado en caso de contarse con centroides, es el índice de Davies–Bouldin este se define con la siguiente fórmula:

$$DB = \frac{1}{n}\sum_{i=1}^{n} \max_{j \neq i} \left(\frac{\sigma_i + \sigma_j}{d(c_i, c_j)}\right)$$

Donde valores menores representan clusters de mejor calidad; Y σ_i representa la similitud entre los puntos del cluster i, este se calcula con la fórmula:

$$\sigma_i = \frac{1}{|C_i|}\sum_{j \in C_i} d(\text{centroid}_i, j)$$

. Además, la separación entre clusters se calcula con la fórmula:

$$d(C_i, C_j) = d(\text{centroid}_i, \text{centroid}_j)$$

Este método también requiere del uso de distancias, por lo que también es susceptible a la medición de la dimensionalidad.

- **Dunn**: Otro método utilizado es el índice de Dunn el cual se centra en identificar clusters densos y bien separados, para esto primero calcula la similitud intra-cluster, esto lo hace calculando, para cada cluster, la distancia existente entre cada par de puntos, con la ecuación: $\delta_i = \min_{j,k \in C_i, j \neq k} d(j,k)$, el valor cuantifica cuan compacto es el cluster. La dis-similitud entre clusters, se cuantifica con la distancia entre los centroides de cada par de clusters, con la con la ecuación: $\Delta_{ij} = d(\text{centroid}_i, \text{centroid}_j)$. Finalmente el índice es la división entre el δ mínimo calculado y el máximo Δ, en otras palabras, se busca maximizar la ecuacion.

$$\text{DI} = \frac{\min_{i \neq j} \Delta_{ij}}{\max_i \delta_i}$$

Este índice es también dependiente del cálculo de distancia entre puntos, por lo tanto es también afectado por la maldición de la dimensionalidad.

- **Calinski–Harabasz**: Otro índice que cuantifica la calidad de clusters, es el índice de, el cual es la división entre la varianza de las distancias entre centroides de clusters, sobre la varianza entre la distancia intra clusters. En este caso valores altos sugieren cluster compactos y bien separados. Como en los casos anteriores, también se depende de distancias y es por lo tanto susceptible a la maldición de la dimensionalidad.

En caso de conocerse las etiquetas reales de los datos, se pueden utilizar otros métodos para evaluar la calidad de los clusters identificados, además de la veracidad de la pertenencia de los datos al cluster específico. Esto es útil en el trabajo realizado, ya que no necesariamente se desea que los datos provenientes de una persona estén en más de un cluster. De esto se introducen algunos métodos.

- Unsupervised Clustering Accuracy (ACC): Es una métrica cuyo valor es la proporción de valores en que el proceso de clustering logró encontrar el cluster correcto. se calcula con la fórmula :

$$ACC = \max_{\pi} \frac{1}{N} \sum_{i=1}^{N} \mathbb{1}\{l_i = \pi(c_i)\}$$

  Siendo N el tamaño de la muestra, l_i la etiqueta correcta del dato i y c_i el cluster asignado. π es una función que mapea, esto es necesario ya que es normal que el clusterizado genere grupos distintos a los definidos por las etiquetas. Y por último la función $l\{\}$ retorna 0 o 1 dependiendo de la veracidad de la expresión sido comprobada

- **Normalized Mutual Information** (NMI): Métrica que calcula el grado de similitud entre clusters propuestos y las etiquetas originales, para esto se calcula la información mutua entre las etiquetas reales y las asignadas al clusterizar, este cálculo informa la reducción de incertidumbre producida en la predicción de la etiqueta real, al conocerse la etiqueta asignada por el modelo. En otras palabras mide la dependencia de ambas variables. Luego esto se divide por la entropía de ambas variables, la fórmula de todo esto es:

$$\mathbf{NMI}(C, L) = \frac{I(C; L)}{\sqrt{H(C) \cdot H(L)}}$$

  Siendo $Claelclusterasignado$,L$laetiquetareal$,I(C;L), el cálculo de la información mutua y tanto $H(C)$ como $H(L)$ son los cálculos de entropía de ambos conjuntos.

2.6.5. Deep Clustering

Deep clustering se refiere al uso de deep learning en el proceso de clustering, en particular se refiere al uso de redes neuronales para aprender automáticamente features en los datos, los cuales son luego clusterizados, todo esto de forma no supervisada.

2.6.6. Autoencoder

Autoencoder (AE) es una clase de red neuronal que tiene como objetivo el aprender una forma eficiente de codificar información de manera eficiente datos sin etiquetar. Se divide en 3 partes, el encoder que transforma los datos ingresados y retorna una versión codificada (representación latente), las cual contiene toda la información original pero de una forma más compacta, el decodificador, el cual toma el output del codificador y retorna la función original y por último se tiene la función de "loss", la cual compara la información original vs el output del decodificador, siendo importante esta función para el entrenamiento del auto encoder ya que esta permite una modificación del modelo en caso de buscarse una modificación de los datos al pasar por el modelo, como podría ser, por ejemplo la eliminación de ruido de datos *eeg*.

En el trabajo realizado el uso de autoencoders ayuda en la reducción de dimensionalidad de los datos y en la reducción de ruido existente en los datos, existen distintas variaciones que traen entre las cuales se destacan:

- **AE**: La versión más simple, utiliza DBN, tanto para el encoder como para el decoder. Esto tiene como desventaja pero tiene dificultades para hacer "feature learning" [15].
- **Convolutional Autoencoder** (CAE): las redes convolucionales pueden conservar la estructura local de los datos al desarrollar las representaciones latentes, mejorando así la extracción de features. Dado esto se definen los Convolutional Autoencoders (CAEs), estos autoencoders utilizan las capas de convolución y agrupamiento y a modo de decoder se utilizan capas convolucionales transpuestas, las cuales realizan la operación inversa a la convolución.
- **Variational AutoEncoders** (vae): es un tipo de autoencoder en el cual el encoder trata de "aprender" la distribución probabilística que distribuye los datos originales y el decoder "genera" datos en dicha distribución, esta clase de autoencoder asume una distribución distribución gaussiana isotrópica tanto en los datos iniciales como en el vector latente resultante del encoder. Esta distribución tiene el requisito de tener su matriz de varianza definida por $\Sigma = \sigma^2 I$ siendo σ la varianza e I la matriz identidad. Esta clase de autoencoder tiene la ventaja de poder generar datos y son usados en la detección de anomalías, pero tienen la desventaja de ser computacionalmente demandantes.
- **LSTM-AE**: Utiliza redes LSTM en el en el encoder y decoder, tiene la ventaja de poder trabajar con inputs de tamaño variable retornando vectores de tamaño constante y gracias al uso de redes LSTM tiene mejores resultados en el trabajo de series de tiempo al ser comparado con VAE o CAE.
- **Adversarial Autoencoder** (AAE): Genera datos utilizando redes generativas adversativas.
- **Denoising autoencoder** (DAE): Autoencoders en los que al ser entrenados se introduce ruido en los datos para entrenar la eliminación de ruido por parte del modelo. Una variante de este es el Convolutional denoising autoencoder (CDAE), el cual puede ser usado en la reducción de ruido en imágenes.

- **Stacked** (AE):Si se quiere aprovechar las ventajas de diferentes AEs, se puede apilar autoencoders, esto se hace pasando los datos por AEs, para luego usar los datos resultantes junto con los originales en los sucesivos AEs. Un uso lógico de esto es iniciar con un DAE combatiendo el ruido de datos, para luego pasar los datos por otro AE.

2.6.7. Enfoques de deep-Clustering

A nivel de arquitectura de redes neuronales existen 2 enfoques en la realización de deep-clustering, primero esta la opcion de construir un pipeline de modelos en que cada uno realice una parte necesaria para la realización del clustering, esto requiere de extraer features de los datos comúnmente utilizando autoencoders u otras variantes de redes neuronales. Luego se realiza el clustering con algoritmos de clustering. El otro enfoque presente es la integración tanto de la extracción de features como del clustering en un mismo modelo.

El caso de trabajar con un único modelo se divide a su vez de acuerdo a la manera que se define su función de loss; de manera general esta se define de la forma:

$$L = \lambda L_n + (1 - \lambda) L_c$$

Donde L_n es la función de loss de la red, la que se encarga de realizar "feature learning" sobre los datos, L_c es la función de loss de clustering, la cual se encarga de mejorar la habilidad de discriminar de los puntos de los atributos aprendidos o que formen grupos /citea y por último $\lambda \in [0, 1]$ es un hiper parámetro que controla el balance entre ambas funciones de loss.

Algunas variaciones de este modelo son:

- **AE-BASED DEEP CLUSTERING**: Esta clase de red utiliza un autoencoder en el aprendizaje de features y una función de clustering al hacer clustering, la función de loss de la red es la función de loss de reconstrucción asociada al autoencoder, por lo que la función de loss de esta clase de red sera de la forma:

  $$L = \lambda L_{recc} + (1 - \lambda) L_c$$

  Existen numerosas representantes de esta clase de red; algunos ejemplos incluyen:

  - **Deep Clustering Network** (DCN): Esta red utiliza un autoencoder tradicional y k-means, el autoencoder es pre-entrenado y luego en el entrenamiento principal, se optimiza la función de loss completa, esto es se tanto el resultado de la reconstrucción del encoder como los resultados del clustering.
  - **Deep Subspace Clustering Networks** (DSC-Nets): Esta red introduce un autoencoder diseñado con el propósito de realizar clustering de subespacios, con este propósito se introduce una capa de "auto expresión" entre el encoder y el decoder, esta capa está completamente conectada no tiene "bias" ni función no lineal de activacion [**?**].

- **Deep Continuous Clustering** (DCC): Utiliza un algoritmo basado en autoencoder para construir una representación continua de los datos. Además no requiere de conocer el número de clusters reales de antemano.

- **CDNN-BASED DEEP CLUSTERING**: Estas redes remplazan el uso de autoencoder utilizados anteriormente por "deep neural networks" las cuales comparten la función de loss con las funciones de clustering, es por esto que la función de loss dera definida por:

$$L = L_c$$

Al eliminar la función de loss de reconstrucción se introduce el riesgo de obtener un espacio de atributos corrupto, perdiendo así el significado de estos.Es por esto que las redes basadas en CDNN tendrán tanto su función de loss como las inicialización de las redes son diseñadas considerando esta restricción.

Una implementación de esta clase de red es el caso de del algoritmo **Mutual Information Maximization** (IMSAT), el cual entrena un clasificador estadístico, el cual busca maximizar la información mutua entre el input y la asignación de cluster (maximizar la dependencia estadística de ambas variables). Además este algoritmo aumenta la cantidad de datos al entrenar realizando "**Self-Augmented Training** ", esto se realiza ingresando datos sin etiquetar al algoritmo, esto datos se generan introduciendo ruido o trasladando datos a clusterziar.

Capítulo 3

Enfoque

El fin de este trabajo es el encontrar grupos de personas, las cuales puedan como partir partes resultados de su entrenamiento, con el propósito de reducir el tiempo requerido para que nuevas personas puedan ejecutar comandos mentales. para lograr esto se requieren lograr encontrar grupos de personas que puedan "utilizar" el entrenamiento de otras personas y además se requiere clasificar nuevas personas en grupos de forma tal que se pueda apreciar una diferencia sustantiva en el tiempo requerido para el entrenamiento.

Para este trabajo se utilizó un dispositivo *Epoch +*, el cual permite, a través de una API, el entrenamiento de comandos virtuales. Los comandos que se entrenaron fueron *"Empujar"*, *"Levantar"*, *"Derecha"* e *"Izquierda"*. Además de estos, también se entrenó el comando *"Neutro"*, el cual tiene como único objetivo establecer una base para el entrenamiento de cada persona. El entrenamiento consiste en la visualización del comando durante 8 segundos. Los resultados de este entrenamiento son guardados por los fabricantes del dispositivo. A su vez, el fabricante proporciona acceso al "perfil.ªsociado al entrenamiento, el cual permite, en caso de estar bien entrenado, detectar los comandos entrenados mientras el usuario piensa en ellos.

Cada persona entrenó los 4 comandos principales 30 veces y el comando neutral 25 veces, lo que resulta en un total de 145 instancias de entrenamiento o 120 si se excluye el comando *"Neutro"*. Cada entrenamiento está numerado, lo que define "rondas" y permite diferenciar cada sesión de entrenamiento de una persona, así como ordenarlas temporalmente.

La aplicación utilizada para ejecutar el entrenamiento guarda al terminar este entrenamiento, numerosos "streams" de datos, de los cuales se utilizaron 2, los datos crudos de la *Electroencefalografía* (*EEG*) y los datos de las *Bandas de Poder* (*POW*), los datos EEG, generan 14 columnas (una por cada sensor) y entre 1065 y 1080 filas por entrenamiento. los datos POW por otra parte generan por cada uno de los sensores 5 datos, estos refiriéndose a ondas de distintas frecuencias que es posible extraer de los datos EEG (ondas alfa, betaL, betaH, gamma y theta). Esto resulta en 70 columnas y entre 65 y 79 filas por cada entrenamiento.

Con el fin de de trabajar con datos de tamaño constante se descartan datos hasta llegar a la cantidad mínima de datos. Los datos eliminados, son los primeros n datos, esto se hace

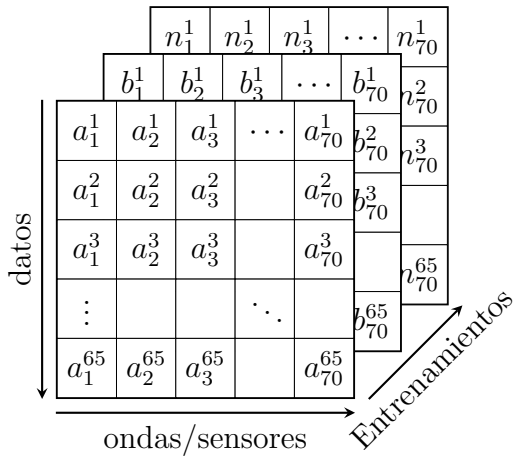


Figura 3.1: Representación de entrenamientos

Id de datos	Band Power	EEG
Persona	(120, 65, 70)	(120, 1038, 14)
Persona-Comando	(30, 65, 70)	(30, 1038, 14)
Persona-Ronda	(65, 70)	(1038, 14)

Tabla 3.1: Distintas formas de agrupar datos

así, debido que de existir datos que no son útiles estos estarán al principio, esto se hace así con el propósito de reducir efectos provocados durante la reacción provocada en la mente durante cambios de comando. Se decidió no extender la cantidad de datos eliminados ya que el diseño de la aplicación utilizada para el entrenamiento, ordena los comandos entrenados de forma tal que entre entrenamientos de comandos distintos, se entrena el comando neutral, el que tiene un efecto de separación entre comandos.

Con el fin de trabajar con datos de tamaño constante, se descartan datos hasta alcanzar la cantidad mínima necesaria. Los datos eliminados corresponden a las primeras n filas. Esto se hace para reducir los efectos provocados durante la reacción mental durante cambios de comando. Se decidió no extender la cantidad de datos eliminados, dado que el diseño de la aplicación utilizada ordena los comandos de manera que entre entrenamientos de comandos distintos se entrena el comando *"Neutro"*, actuando como una separación entre comandos.

Sumándose a la normalización de la cantidad de datos, también se excluyó el comando *"Neutro"* debido a que fue entrenado con menor frecuencia. Por lo tanto, al finalizar el experimento, se define a cada participante como una matriz tridimensional de tamaño (120, 65, 70) desde el punto de vista de los datos POW, y de tamaño (120, 1038, 14) para los datos EEG.

Como se habló en la Sección 2.6.1, la maldición de la dimensionalidad es un problema que existe al trabajar con datos de alta dimensionalidad, como es el caso de los datos recién descritos. Es por esto que se planea subdividir los datos asociados a personas, tanto por comando como por ronda, de acuerdo a lo presentado en la Tabla 3.1, explorando en la búsqueda de resultados. Esto se suma a las técnicas de reducción de dimensionalidad antes descritas.

Es importante destacar que originalmente participaron 30 personas en el estudio. Sin embargo, fue necesario excluir a un participante debido a que no completó el programa de entrenamiento y, por otro lado, debido a complicaciones relacionadas con el uso de una licencia requerida para recopilar datos *EEG*, solo se tienen datos de este tipo de 20 participantes. Por lo tanto, utilizando los datos de bandas de potencia, se definen 29 personas, mientras que con los datos *EEG* se definen 20.

Capítulo 4

Adquisición de datos

4.1. Aplicación

Dado el propósito de la investigación, i.e buscar relaciones entre ondas cerebrales de distintas personas, las cuales permiten "compartir" parte del entrenamiento. Esta selección ocurriendo durante el proceso de entrenamiento de diferentes comandos mentales; Se seleccionaron una serie de requisitos que los datos deberían cumplir para que el proceso tenga alguna posibilidad de introducir ventajas por sobre el proceso normal realizado automáticamente por los servidores de *Emotiv*. Esto requisitos son:

- Corresponder a señales *eeg* asociadas a entrenamiento de comandos mentales, esto ya que se quiere poder asociar a personas con mas una otra persona en caso de que diferentes comandos resulten asociaciones a distintas personas.
- Capturar datos a tiempo real durante el uso del dispositivo, de forma automática, posibilitandose la ejecución de modelos que permitan la agrupación de distintos participantes del entrenamiento.

Emotiv, provee distintas aplicaciones diseñadas para ser utilizadas en conjunto con el dispositivo *BCI*, estas aplicaciones ofrecen distintas fracciones de los requisitos planteados, pero ninguna cumple con el conjunto completo, esto ya que la aplicación que permite realizar el entrenamiento no tiene la opción de guardar los datos generados durante el entrenamiento, existe una aplicación enfocada en guardar los datos registrados por el dispositivo *BCI*, pero la conexión de múltiples aplicación a un mismo dispositivo de forma paralela no está permitido. E incluso de permitirse esto, la coordinación entre ambas aplicaciones e inexistente, por todo esto es que se decidió implementar una aplicación utilizando la *api* otorgada por *Emotiv*.

La aplicación desarrollada implementa 7 comandos mentales que pueden ser entrenados por los usuarios, siendo estos *"Neutro"*, *"Empujar"*, *"Tirar"*, *"Levantar"*, *"Caer"*, *"Derecha"* e *"Izquierda"*. Cada uno de estos comandos se representa visualmente por medio de un cuadrado que realiza el movimiento descrito en pantalla durante 8 segundos, a excepción del comando *"Neutro"*, el cual se representa mediante un mensaje que invita al usuario a

descansar la mente durante el mismo intervalo de tiempo. Dado que *Emotiv* permite entrenar únicamente 5 comandos por perfil de usuario, durante la prueba de la aplicación solo se entrenaron los comandos *"Neutro"*, *"Empujar"*, *"Levantar"*, *"Derecha"* e *"Izquierda"*. La inclusión del comando *"Neutro"* no solo tiene como propósito brindar un descanso al usuario durante el entrenamiento, sino también cumplir con un requisito específico de *Emotiv*, que exige su presencia para validar el perfil utilizado.

La aplicación fue escrita en python y se divide en 2 partes, una parte se encarga de dibujar en pantalla la representación visual asociada al comando siendo entrenado y de definir el siguiente comando a entrenar en caso que esto se requiera. La otra parte se encarga de mantener la comunicación con el dispositivo *BCI* tanto para comunicar el comando a entrenar como para registrar los datos recibidos por este, catalogando el participante, el comando entrenado y la ronda correspondiente (contador que incrementa con cada instancia de comando entrenado). Ambas partes son ejecutadas en threads separados y se comunican entre si utilizando la librería "*queue*" de python, la cual implanta estructuras de tipo "*FIFO*" seguras para ser usadas por distintos *threads*. El funcionamiento de ambas partes sera definido en mayor detalle en las secciones siguientes.

El uso habitual de la aplicación inicia con un mensaje de bienvenida, el cual espera la confirmación del usuario para comenzar el proceso de entrenamiento. Una vez confirmado, se da inicio al entrenamiento, que comprende 30 instancias para cada uno de los 4 comandos entrenados, excluyendo el comando *"Neutro"*. Estas instancias se dividen en grupos de 5 para cada comando. Después de cada grupo, se realiza un receso"donde se entrena el comando *"Neutro"*, seguido de otro grupo para el siguiente comando deseado. Este ciclo continúa hasta completar el número deseado de entrenamientos, momento en el cual se muestra un mensaje de despedida y la aplicación se cierra automáticamente. El proceso completo tiene una duración de 20 minutos.

4.1.1. Parte visual de la aplicación

Este es el thread principal de la aplicación y se encarga de inicializar el otro thread, de definir el orden de los comandos a entrenar y de la cantidad de rondas que serán entrenadas. Además se preocupa de mantener la coordinación con la parte preocupada de comunicarse con el dispositivo *BCI* asegurando así que el entrenamiento realizado este sincronizado con las animaciones.

La interfaz gráfica de la aplicación se realizo utilizando la librería *tkinter* de python, con esta libreria se crea la ventana, se dibuja la figura que hace de guía para el entrenamiento y se realiza las animaciones pertinentes.

4.1.2. Parte Encargada de entrenar comandos

Esta parte se encarga de realizar todos los pasos requeridos por *Emotiv* en el uso del dispositivo *BCI*, incluyendo pedir acceso a la api, autenticar el usuario, crear la sesión, susbcrirse a los streams de datos, etc. Luego se encarga de leer los comandos mentales a entrenar, para

luego inicializar dicho entrenamiento guardando además los datos recibidos por el dispositivo.

Emotiv mantiene código de ejemplo [1], código perteneciente a este repositorio fue utilizado en la construcción de la aplicación, teniéndose ya implementado los pasos requeridos para utilizar el dispositivo *BCI* y conectarse a los servidores de *Emotiv*. Sin embargo, fue necesario modificar su funcionamiento con el propósito de elegir el comando a entrenar de forma dinámica y además se modificó la manera en que se guardan streams de datos.

4.1.3. Recolección de los datos

Se realizó un llamado a participar a través de la plataforma digital de *"U-Cursos"* en la prueba de la aplicación, con el propósito de recolectar datos. Gracias a esto, 30 personas participaron en el estudio, de las cuales 25 declararon pertenecer al género masculino, 4 al género femenino y una se identificó como no binaria. Del grupo completo, 28 son estudiantes de la Facultad de Ciencias Físicas y Matemáticas de la Universidad de Chile; de los otros 2, uno trabaja como funcionario en la misma facultad y el otro es estudiante de otra facultad de la misma universidad.

Cada participante inició la prueba leyendo y firmando un formulario de consentimiento informado. Luego, se le colocó el dispositivo *BCI* sobre la cabeza, siguiendo las instrucciones oficiales de *Emotiv*. A continuación, se llevó a cabo el proceso de entrenamiento con la aplicación implementada. Finalmente, en 7 de los 30 participantes, se realizó un entrenamiento de los comandos *"push"* y *"lift"* 10 veces cada uno, utilizando el software oficial de *Emotiv*, con el propósito de comparar los resultados obtenidos entre ambas aplicaciones. En total cada participante demoro entre 45 y 60 minutos en realizar el proceso completo.

4.1.4. Resultados de la prueba

Durante la prueba se noto una diferencia entre la calidad de entrenamiento de la aplicación desarrollada en comparación, esta diferencia puede ser explicada debido a la existencia de requisitos contradictorios a la hora de diseñar la aplicación. Primeramente se busco maximizar el tiempo en que los usuarios utilicen el dispositivo *BCI*, sin distraerse al momento de entrenar comandos, por lo que se incluyeron la mayor cantidad de comandos distintos posibles dada la limitada cantidad permitida por los perfiles asociados. Lo que crea un conflicto con el correcto entrenamiento de comandos mentales, para los cuales *Emotiv* recomienda fuertemente entrenar una menor cantidad y agregar nuevos comandos si y solo cuando ya se tienen los anteriores satisfactoriamente entrenados.

El efecto de aumentar la cantidad de comandos entrenados puede observarse en la Figura 4.1, donde se muestra el promedio de tiempo en que el dispositivo detectó el comando correcto durante el entrenamiento (promedio tomando en cuenta todos los participantes). En este gráfico, también se aprecia que cada 5 rondas, la proporción es 0; esto representa la forma en que se lleva a cabo el entrenamiento del comando *"Neutro"*.

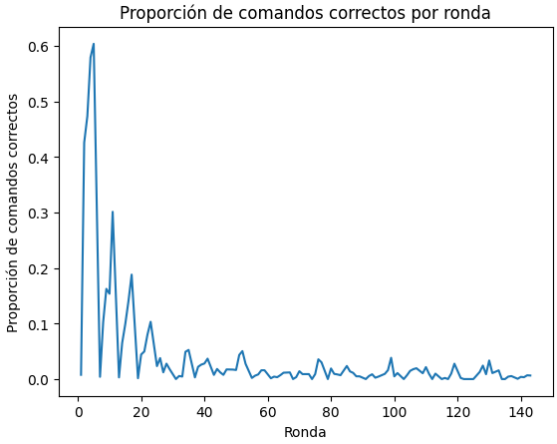


Figura 4.1: Promedio de proporción de comandos correctos por ronda

4.1.5. Diferencia entre aplicación y software de *Emotiv*

Además de incluir la habilidad de entrenar comandos mentales mientras se guarda la representación de las ondas mentales, existen algunas diferencias entre la aplicación, la mas importante es la carencia de capacidad de eliminar instancias de entrenamiento en caso de ser asignado un "mal" puntaje, por parte de emotiv. Este puntaje se presenta después de realizar cada instancia de entrenamiento si es que que se cuentan con 3 instancias guardadas en el perfil de *Emotiv*. El puntaje tiene un valor numérico de 1 a 100 y en caso de se un valor "bajo", se recomienda eliminar la instancia de entrenamiento, siendo esto equivalente a no haber realizado el entrenamiento.

La otra gran diferencia es la libertad con la que se permite elegir el comando mental a ser entrenado. mientras el software permite elegir cada comando en el momento, la aplicación desarrollada sigue una secuencia predefinida de comandos, la cual es idéntica cada vez.

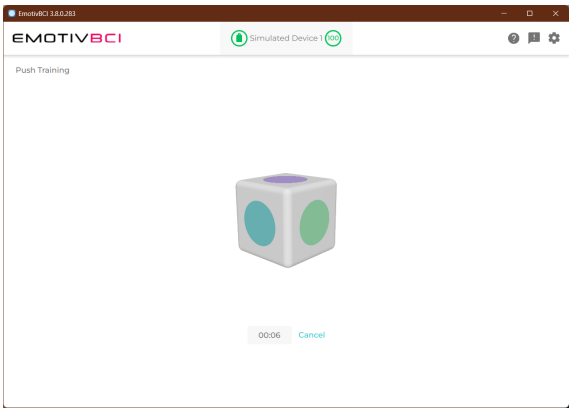


Figura 4.2: Entrenamiento de comando mental a través de *EnotivBCI*.



Figura 4.3: Entrenamiento de comando mental a través de aplicación desarrollada.

Figura 4.4: Comparación visual de aplicaciones de entrenamiento.

Capítulo 5

Análisis

5.1. Búsqueda clusters

5.1.1. Reducción de dimensiones con Autoencoder

Lo primero que se intentó fue el entrenamiento de un autoencoder simple. Su encoder consiste en 3 capas lineales que reducen la dimensión de los datos hasta solo 32, mientras que el decoder tiene la misma arquitectura pero de forma contraria. Posteriormente, se redujo la dimensionalidad de la información codificada utilizando PCA.

Los resultados de esto pueden verse en las figuras es que no se aprecian agrupaciones que sugieran solucionar el problema descrito.

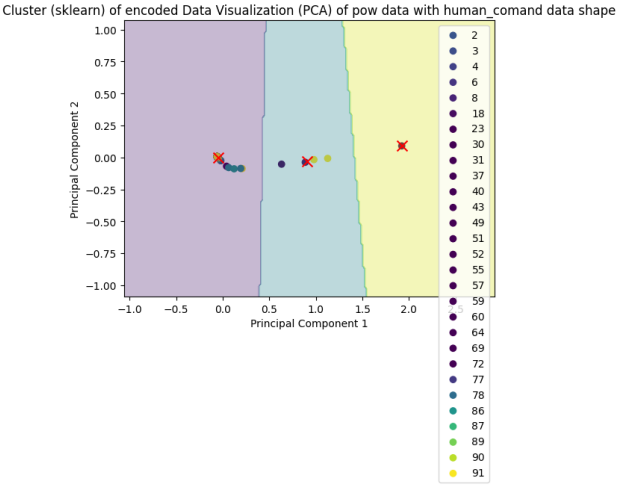


Figura 5.1: Resultado reducción dimensionalidad con AE y PCA sobre datos *pow* utilizando datos de la forma humano-comando

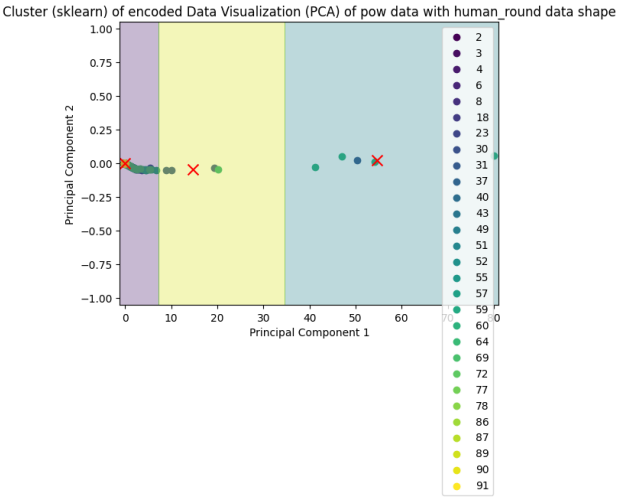


Figura 5.2: Resultado reducción dimensionalidad con AE y PCA sobre datos *pow* utilizando datos de la forma humano-ronda

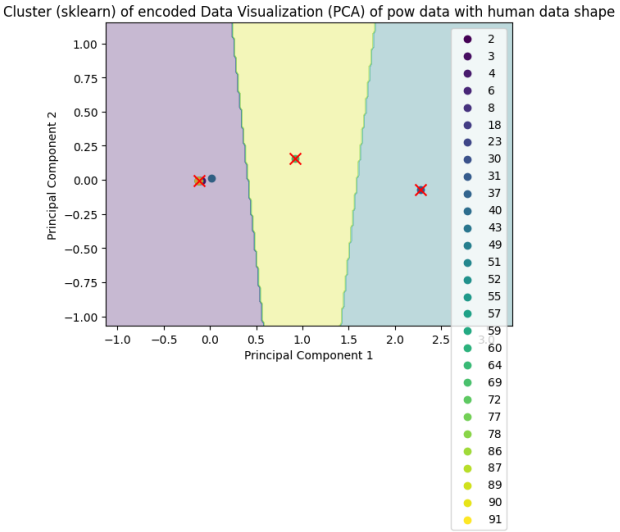


Figura 5.3: Resultado reducción dimensionalidad con AE y PCA sobre datos pow utilizando datos de la forma humano *pow*

Capítulo 6

Conclusión

El trabajo realizado contempla la implementación de una aplicación que permite entrenar comandos mentales utilizando el dispositivo *BCI*, modelo *Epoch +* de la marca *Emotiv*. Además, se inició un trabajo exploratorio en la búsqueda de agrupaciones de personas que permitieran utilizar el entrenamiento de una persona como punto de partida para otra. Sin embargo, hasta el momento, no se han encontrado formas efectivas de realizar estas agrupaciones.

Bibliografía

[1] Example with cortex v2/v3 api.

[2] SEED Dataset.

[3] tkinter—python interface to tcl/tk.

[4] Salma Alhagry, Aly Aly Fahmy, and Reda A El-Khoribi. Emotion recognition based on eeg using lstm recurrent neural network. *International Journal of Advanced Computer Science and Applications*, 8(10), 2017.

[5] Mahnaz Arvaneh, Ian Robertson, and Tomas E. Ward. Subject-to-subject adaptation to reduce calibration time in motor imagery-based brain-computer interface. In *2014 36th Annual International Conference of the IEEE Engineering in Medicine and Biology Society*, pages 6501–6504, 2014.

[6] Ira Assent. Clustering high dimensional data. *WIREs Data Mining and Knowledge Discovery*, 2(4):340–350, 2012.

[7] Alexander Craik, Yongtian He, and Jose L Contreras-Vidal. Deep learning for electroencephalogram (eeg) classification tasks: a review. *Journal of Neural Engineering*, 16(3):031001, apr 2019.

[8] Chenglong Dai, Jia Wu, Dechang Pi, Stefanie I. Becker, Lin Cui, Qin Zhang, and Blake Johnson. Brain EEG Time-Series Clustering Using Maximum-Weight Clique. *IEEE Transactions on Cybernetics*, 52(1):357–371, January 2022. Conference Name: IEEE Transactions on Cybernetics.

[9] Maryam Dehghani, Ali Mobaien, and Reza Boostani. A deep neural network-based transfer learning to enhance the performance and learning speed of bci systems. *Brain-Computer Interfaces*, 8(1-2):14–25, 2021.

[10] EMOTIV. Dispositivo eeg epoc-x, emotic.

[11] EMOTIV. Documentación cortex.

[12] Fatemeh Fahimi, Zhuo Zhang, Wooi Boon Goh, Tih-Shi Lee, Kai Keng Ang, and Cuntai Guan. Inter-subject transfer learning with an end-to-end deep convolutional neural network for eeg-based bci. *Journal of Neural Engineering*, 16(2):026007, jan 2019.

[13] Nikita Gurudath and H. Bryan Riley. Drowsy driving detection by eeg analysis using wavelet transform and k-means clustering. *Procedia Computer Science*, 34:400–409, 2014. The 9th International Conference on Future Networks and Communications (FNC'14)/The 11th International Conference on Mobile Systems and Pervasive Computing (MobiSPC'14)/Affiliated Workshops.

[14] Vinay Jayaram, Morteza Alamgir, Yasemin Altun, Bernhard Schölkopf, and Moritz Grosse-Wentrup. Transfer learning in brain-computer interfaces. *IEEE Computational Intelligence Magazine*, 11:20–31, 2015.

[15] Md Rezaul Karim, Oya Beyan, Achille Zappa, Ivan G Costa, Dietrich Rebholz-Schuhmann, Michael Cochez, and Stefan Decker. Deep learning-based clustering approaches for bioinformatics. *Briefings in Bioinformatics*, 22(1):393–415, January 2021.

[16] Stamos Katsigiannis and Naeem Ramzan. DREAMER: A Database for Emotion Recognition Through EEG and ECG Signals From Wireless Low-cost Off-the-Shelf Devices. *IEEE Journal of Biomedical and Health Informatics*, 22(1):98–107, January 2018. Conference Name: IEEE Journal of Biomedical and Health Informatics.

[17] Matthias Krauledat, Michael Schröder, Benjamin Blankertz, and Klaus-Robert Müller. Reducing calibration time for brain-computer interfaces: A clustering approach. In B. Schölkopf, J. Platt, and T. Hoffman, editors, *Advances in Neural Information Processing Systems*, volume 19. MIT Press, 2006.

[18] Xujin Li, Wei Wei, Shuang Qiu, and Huiguang He. Tff-former: Temporal-frequency fusion transformer for zero-training decoding of two bci tasks. In *Proceedings of the 30th ACM International Conference on Multimedia*, MM '22, page 51–59, New York, NY, USA, 2022. Association for Computing Machinery.

[19] Mamta Mittal, Lalit M. Goyal, Duraisamy Jude Hemanth, and Jasleen K. Sethi. Clustering approaches for high-dimensional databases: A review. *WIREs Data Mining and Knowledge Discovery*, 9(3):e1300, 2019. _eprint: https://onlinelibrary.wiley.com/doi/pdf/10.1002/widm.1300.

[20] Jelena Mladenović. Standardization of protocol design for user training in eeg-based brain–computer interface. *Journal of Neural Engineering*, 18(1):011003, feb 2021.

[21] M. F. Mridha, Sujoy Chandra Das, Muhammad Mohsin Kabir, Aklima Akter Lima, Md. Rashedul Islam, and Yutaka Watanobe. Brain-computer interface: Advancement and challenges. *Sensors*, 21(17), 2021.

[22] Umut Orhan, Mahmut Hekim, and Mahmut Ozer. EEG signals classification using the K-means clustering and a multilayer perceptron neural network model. *Expert Systems with Applications*, 38(10):13475–13481, 2011.

[23] Lance Parsons, Ehtesham Haque, and Huan Liu. Subspace clustering for high dimensional data: a review. *ACM SIGKDD Explorations Newsletter*, 6(1):90–105, June 2004.

[24] Aya Rezeika, Mihaly Benda, Piotr Stawicki, Felix Gembler, Abdul Saboor, and Ivan Volosyak. Brain–computer interface spellers: A review. *Brain sciences*, 8(4):57, 2018.

[25] Simanto Saha and Mathias Baumert. Intra- and inter-subject variability in eeg-based sensorimotor brain computer interface: A review. *Frontiers in Computational Neuroscience*, 13, 2020.

[26] Simanto Saha, Khondaker A. Mamun, Khawza Ahmed, Raqibul Mostafa, Ganesh R. Naik, Sam Darvishi, Ahsan H. Khandoker, and Mathias Baumert. Progress in brain computer interface: Challenges and opportunities. *Frontiers in Systems Neuroscience*, 15, 2021.

[27] Alex Sherstinsky. Fundamentals of recurrent neural network (rnn) and long short-term memory (lstm) network. *Physica D: Nonlinear Phenomena*, 404:132306, 2020.

[28] Jerry J. Shih, Dean J. Krusienski, and Jonathan R. Wolpaw. Brain-computer interfaces in medicine. *Mayo Clinic Proceedings*, 87(3):268–279, 2012.

[29] Revati Shriram, Mahalingam Sundhararajan, and Nivedita Daimiwal. Eeg based cognitive workload assessment for maximum efficiency. *IOSR Journal of Electronics and Communication Engineering (IOSR-JECE)*, 08 2012.

[30] Jiayao Sun, Jin Xie, and Huihui Zhou. Eeg classification with transformer-based models. In *2021 IEEE 3rd Global Conference on Life Sciences and Technologies (LifeTech)*, pages 92–93, 2021.

[31] Gabriel Alves Mendes Vasiljevic and Leonardo Cunha de Miranda. Brain–computer interface games based on consumer-grade eeg devices: A systematic literature review. *International Journal of Human–Computer Interaction*, 36(2):105–142, 2020.

[32] Francis R. Willett, Donald T. Avansino, Leigh R. Hochberg, Jaimie M. Henderson, and Krishna V. Shenoy. High-performance brain-to-text communication via handwriting. *Nature*, 593(7858):249–254, May 2021.

[33] Jonathan R Wolpaw, Niels Birbaumer, William J Heetderks, Dennis J McFarland, P Hunter Peckham, Gerwin Schalk, Emanuel Donchin, Louis A Quatrano, Charles J Robinson, Theresa M Vaughan, et al. Brain-computer interface technology: a review of the first international meeting. *IEEE transactions on rehabilitation engineering*, 8(2):164–173, 2000.

Apéndice A

Anexo

A.1. Imágenes aplicación entrenamiento

A continuación se tienen imágenes ejemplificando el funcionamiento de la aplicación desarrollada en un caso hipotético en que se entrene una vez cada comando. Partiendo por el mensaje de bienvenida, pasando por cada comando, donde se muestra el inicio y el fin de cada comando cuando es necesario y termina con un mensaje de despedida.

Notese que por restricciones de *Emotiv*, el entrenar mas de 5 comandos mentales no esta permitido en un mismo perfil.

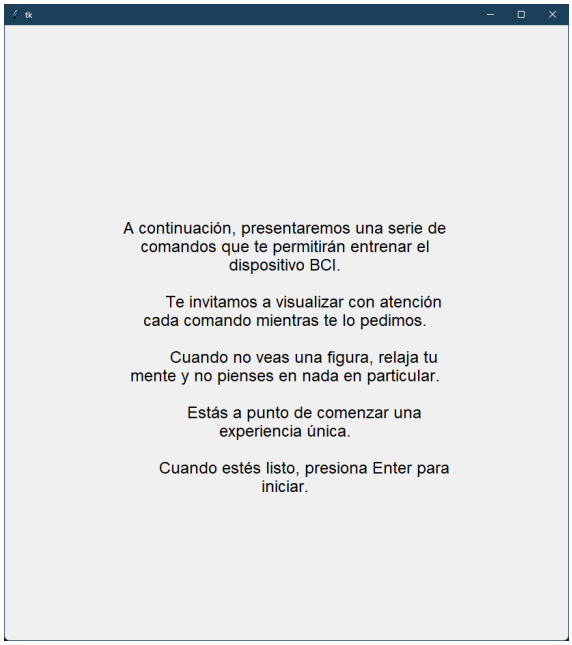


Figura A.1: Mensaje al iniciar la aplicacion

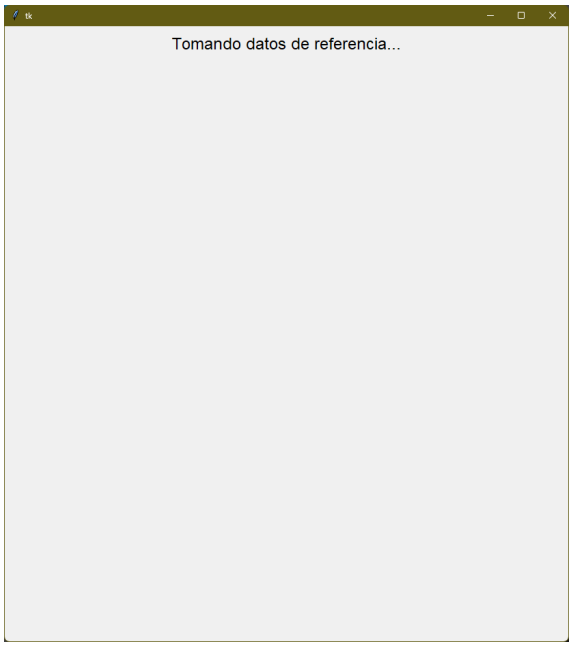


Figura A.2: Mensaje presente al entrenar comando neutral



Figura A.3: inicio y fin de animación presente al entrenar comando *"empujar"*.



Figura A.4: inicio y fin de animación presente al entrenar comando *"tirar"*.



Figura A.5: inicio y fin de animación presente al entrenar comando *"caer"*.



Figura A.6: inicio y fin de animación presente al entrenar comando *"levantar"*.

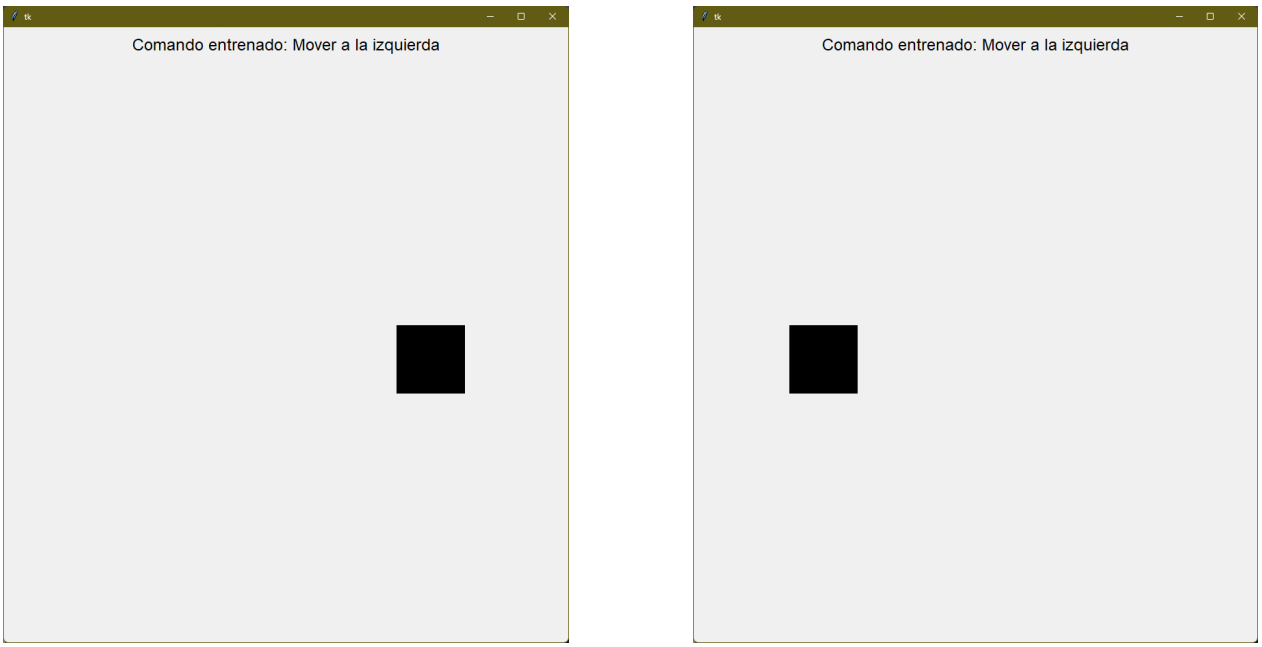


Figura A.7: inicio y fin de animación presente al entrenar comando *"izquierda"*.

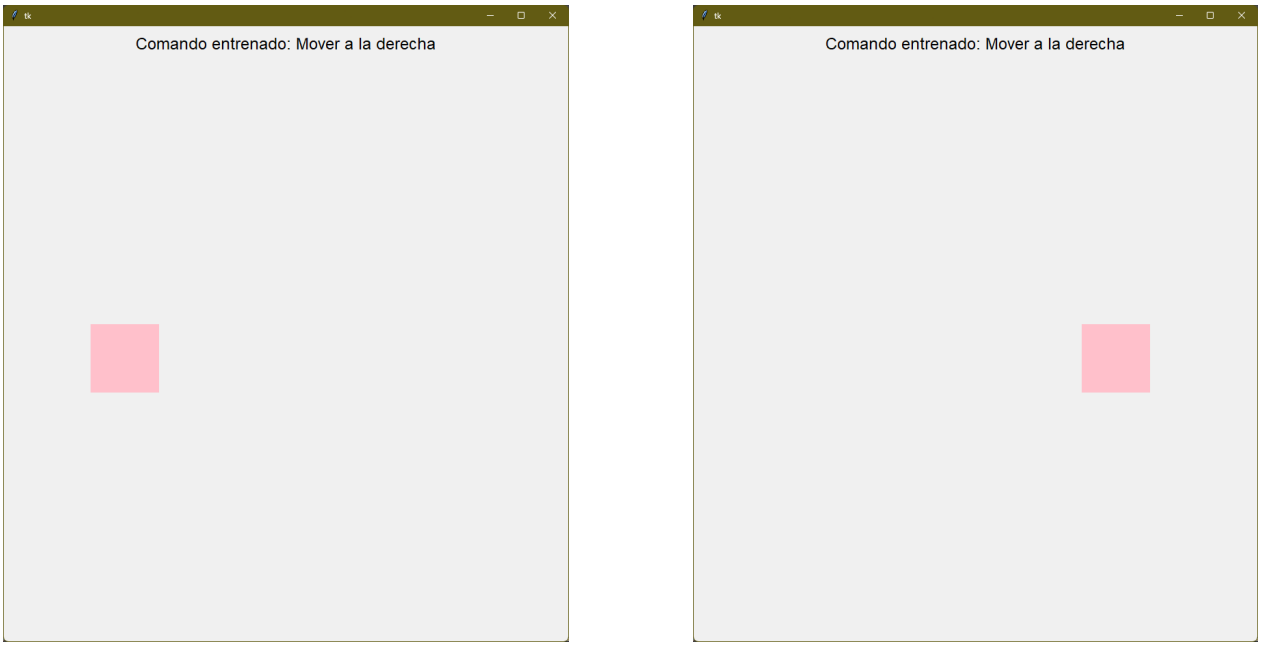


Figura A.8: inicio y fin de animación presente al entrenar comando *"derecha"*.



Figura A.9: Mensaje presente al terminar el proceso